

AI ASSISTED CODING

ASSIGNMENT-12.1

NAME:P.AJAY

ROLL NO:2403A510B4

BATCH:05

DATE:22-09-2025

Task 1 – Merge Sort Implementation

Prompt

Write a Python function `merge_sort(arr)` that sorts a list in ascending order using the Merge Sort algorithm. Include time and space complexity in the docstring. Provide test cases with assertions.

Code

```
def merge_sort(arr):
    """
    Merge Sort Algorithm
    -----
    Time Complexity:
    | Best Case: O(n log n)
    | Average Case: O(n log n)|
    | Worst Case: O(n log n)
    Space Complexity: O(n) [ additional space is used for merging.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    """Helper function to merge two sorted lists."""
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

# ✅ Test Cases
assert merge_sort([5, 3, 8, 1, 2]) == [1, 2, 3, 5, 8]
assert merge_sort([]) == []
assert merge_sort([10]) == [10]

print("All Merge Sort test cases passed!")
```

Output

```
PS C:\Users\AJAY\ai coding\ai coding> & C:/Users/AJAY/AppData/Local/Programs/Python/ai coding/ai coding/12.1.py
All Merge Sort test cases passed!
```

Observation

The AI-generated Merge Sort code works correctly. It passes all 3 assert test cases. Time complexity = $O(n \log n)$ and Space complexity = $O(n)$ as expected.

Task 2 – Binary Search Implementation

Prompt

Write a Python function `binary_search(arr, target)` that returns the index of the target in a sorted list or -1 if not found. Add docstrings explaining best, average, and worst case complexities. Include at least 3 test cases.

Code

```
12.1.py > ...
def binary_search(arr, target):
    """
    Binary Search Algorithm
    -----
    Time Complexity:
        Best Case:  $O(1)$  if the target is found at the mid
        Average Case:  $O(\log n)$ 
        Worst Case:  $O(\log n)$ 
    Space Complexity:  $O(1)$  iterative version uses constant space
    """
    low, high = 0, len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# ✓ Test Cases
assert binary_search([1, 2, 3, 4, 5], 3) == 2
assert binary_search([10, 20, 30, 40, 50], 25) == -1
assert binary_search([], 5) == -1

print("All Binary Search test cases passed!")
```

Output

```
PS C:\Users\AJAY\ai coding\ai coding> & C:/Users/AJAY/AppData/Local/Programs/Python/Python313/python.exe
Y/ai coding/ai coding/12.1.py"
All Binary Search test cases passed!
```

Observation

Binary Search works correctly for found and not-found cases. Passes all assert test cases. Complexity correctly documented: $O(\log n)$ time, $O(1)$ space.

Task 3 – Inventory Management System

Prompt

Suggest the best search and sort algorithms for a retail inventory system (thousands of products). Implement functions for searching by ID/name and sorting by price/quantity. Justify algorithm choice in a table.

Recommendation Table

Operation	Recommended Algorithm	Justification
Search by ID	Hash Map (dict)	$O(1)$ lookup, IDs unique, frequent searches
Search by Name	Binary Search	Works on sorted list of names, $O(\log n)$
Sort by Price/Quantity	Timsort (Python sorted)	Optimized for real datasets, $O(n \log n)$

Code

```
# Inventory represented as a list of dicts
inventory = [
    {"id": 101, "name": "Laptop", "price": 60000, "quantity": 15},
    {"id": 102, "name": "Mouse", "price": 500, "quantity": 150},
    {"id": 103, "name": "Keyboard", "price": 1500, "quantity": 85},
    {"id": 104, "name": "Monitor", "price": 12000, "quantity": 40},
]

# Search by ID using dict (hash map)
def search_by_id(inventory_dict, product_id):
    return inventory_dict.get(product_id, "Not Found")

# Search by name using binary search
def search_by_name(sorted_inventory, target_name):
    low, high = 0, len(sorted_inventory) - 1
    while low <= high:
        mid = (low + high) // 2
        if sorted_inventory[mid]["name"] == target_name:
            return sorted_inventory[mid]
        elif sorted_inventory[mid]["name"] < target_name:
            low = mid + 1
        else:
            high = mid - 1
    return "Not Found"

# Sort by price
def sort_by_price(inventory):
    return sorted(inventory, key=lambda x: x["price"])

# Sort by quantity
def sort_by_quantity(inventory):
    return sorted(inventory, key=lambda x: x["quantity"])

# Build hash map for fast ID lookup
inventory_dict = {item["id"]: item for item in inventory}

# Test Cases
assert search_by_id(inventory_dict, 101)["name"] == "Laptop"
assert search_by_name(sorted(inventory, key=lambda x: x["name"]), "Mouse")["price"] == 500
assert sort_by_price(inventory)[0]["name"] == "Mouse"
assert sort_by_quantity(inventory)[-1]["name"] == "Mouse"

print("All Inventory Management test cases passed!")
```

Output

```
PS C:\Users\AJAY\ai coding\ai coding> & C:/Users/AJAY/AppData/Local/Programs/Python/Python313/python.
ers/AJAY/ai coding/ai coding/12.1.py"
All Inventory Management test cases passed!
```

Observation

Search by ID → $O(1)$ with dict.

Search by Name → $O(\log n)$ with binary search.

Sorting → Uses Python's built-in sorted() (Timsort, $O(n \log n)$).

All assert test cases pass.